

Apartment Management System

Document Version: 2.0

Date: 2026-06-03

1. Project Summary

Item	Description
Project Name	Apartment Management System
Type	Add-on modules to existing project
Goal	Add expense, water billing, maintenance, payments, complaints, notifications
Target Users	Residents (Mobile App) + Admin (Web Dashboard)
Auth Method	Email/Password (already working)

2. Technology Stack

Layer	Technology
Frontend (Resident)	React Native / Flutter (existing)
Frontend (Admin)	React (existing)
Backend	Node.js + Express (existing)
Database	PostgreSQL (existing)
Authentication	JWT (existing)
File Storage	As per existing
PDF Generation	PDFKit / jsPDF

Layer	Technology
Notifications	WhatsApp Business API + Nodemailer

3. Existing System Overview

Already Working

Module	Status
Auth Register/Login (email/password)	✓
Community Management	✓
Dashboard	✓
Flat Management	✓
Resident Management	✓
Service Requests	✓
Vendor Management	✓
Feedback Management	✓
Notifications (basic)	✓

Not Connected (Backend exists, frontend missing)

Module	Action Needed
Audit Logs	Connect frontend
Timeline Page	Connect frontend

4. New Modules Summary

Module	Priority
Expense Management	P0
Water Meter Calculation	P0
Maintenance Billing	P0
Payments Collection	P1
Complaints	P1
Notifications (auto alerts)	P2

5. Database Schema

New Tables

sql

-- 5.1 expenses

```
CREATE TABLE expenses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category VARCHAR(50) NOT NULL,
  expense_date DATE NOT NULL,
  vendor VARCHAR(100),
  amount DECIMAL(12,2) NOT NULL,
  attachment_url TEXT,
  notes TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW()
);
```

-- 5.2 water_readings

```
CREATE TABLE water_readings (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  flat_id UUID REFERENCES flats(id),  
  previous_reading INTEGER NOT NULL,  
  current_reading INTEGER NOT NULL,  
  month VARCHAR(7) NOT NULL,  
  photo_url TEXT,  
  submitted_by UUID REFERENCES users(id),  
  created_at TIMESTAMP DEFAULT NOW(),  
  UNIQUE(flat_id, month)  
);
```

-- 5.3 water_calculations

```
CREATE TABLE water_calculations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  month VARCHAR(7) NOT NULL,  
  total_tanker_cost DECIMAL(12,2),  
  total_usage_liters INTEGER,  
  per_liter_cost DECIMAL(6,4),  
  manjeera_charge DECIMAL(10,2),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- 5.4 bills

CREATE TABLE bills (

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
flat_id UUID REFERENCES flats(id),
month VARCHAR(7) NOT NULL,
due_date DATE NOT NULL,
common_maintenance DECIMAL(10,2) NOT NULL,
water_charge DECIMAL(10,2) NOT NULL,
reserve_fund DECIMAL(10,2) DEFAULT 0,
special_charges DECIMAL(10,2) DEFAULT 0,
arrears DECIMAL(10,2) DEFAULT 0,
total DECIMAL(10,2) NOT NULL,
status VARCHAR(20) DEFAULT 'unpaid',
pdf_url TEXT,
generated_at TIMESTAMP DEFAULT NOW()
```

);

-- 5.5 payments

CREATE TABLE payments (

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
bill_id UUID REFERENCES bills(id),
amount DECIMAL(10,2) NOT NULL,
mode VARCHAR(20) NOT NULL,
transaction_id VARCHAR(100),
paid_date DATE NOT NULL,
recorded_by UUID REFERENCES users(id),
```

```
created_at TIMESTAMP DEFAULT NOW()
);

-- 5.6 complaints

CREATE TABLE complaints (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  flat_id UUID REFERENCES flats(id),
  category VARCHAR(50) NOT NULL,
  description TEXT NOT NULL,
  priority VARCHAR(20) DEFAULT 'medium',
  photo_url TEXT,
  status VARCHAR(20) DEFAULT 'open',
  assigned_to VARCHAR(100),
  remark TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

No changes to users table (no phone column needed)

6. API Endpoints

Base URL: /api/v1

6.1 Auth (Already Working - No Changes)

Method	Endpoint	Status
POST	/auth/register	✓ Working
POST	/auth/login	✓ Working

6.2 Expense Management

Method	Endpoint	Auth
POST	/expenses	Admin
GET	/expenses	Admin
GET	/expenses/:id	Admin
PUT	/expenses/:id	Admin
DELETE	/expenses/:id	Admin
GET	/expenses/categories	All

6.3 Water Meter

Method	Endpoint	Auth
POST	/water/readings	Resident/Admin
GET	/water/readings	Admin
GET	/water/readings/flat/:flatId	Resident/Admin
GET	/water/readings/my	Resident
POST	/water/calculate	Admin
GET	/water/calculation/:month	Admin

6.4 Maintenance Billing

Method	Endpoint	Auth
POST	/bills/generate	Admin

Method	Endpoint	Auth
GET	/bills	Admin
GET	/bills/my	Resident
GET	/bills/:id	Resident/Admin
GET	/bills/:id/pdf	Resident/Admin

6.5 Payments

Method	Endpoint	Auth
POST	/payments	Resident/Admin
GET	/payments/my	Resident
GET	/payments/pending	Admin
GET	/payments/collection-report	Admin

6.6 Complaints

Method	Endpoint	Auth
POST	/complaints	Resident
GET	/complaints	Admin
GET	/complaints/my	Resident
GET	/complaints/:id	Resident/Admin
PUT	/complaints/:id/status	Admin
PUT	/complaints/:id	Resident (own, if open)

6.7 Notifications (Auto-triggered)

Event	Trigger
Bill generated	Admin clicks generate

Event	Trigger
Payment reminder	Daily cron job
Complaint status change	Admin updates status

7. Business Logic

7.1 Water Meter

- Previous reading = last month's current reading
- Cannot enter reading lower than previous
- One reading per flat per month

7.2 Bill Generation

- Generate only once per month
- Common maintenance = Total expenses ÷ Total flats
- Water bill = Usage × PerLiterCost + (Manjeera charge ÷ Total flats)
- Arrears = Previous month unpaid amount

7.3 Payments

- Partial payment allowed
- Bill status = "partial" if amount < total
- Bill status = "paid" if amount ≥ total
- Unpaid bills carry to next month as arrears

7.4 Complaints

- Resident can edit only if status = "open"
 - Admin can update status anytime
 - Status change triggers notification
-

8. Frontend Pages to Build

Resident Mobile App

Page	Description
My Water Meter	View history, submit reading with photo
My Bills	List bills, view PDF, Pay button
Payment History	List past payments
Raise Complaint	Form with category, description, priority, photo
My Complaints	List with status badges

Admin Web Dashboard

Page	Description
Expenses	Add/Edit/Delete, filter by category/date
Water Meters	View readings, enter tanker cost, run calculation
Bill Generation	Select month, set due date, generate
All Bills	Table view with filters
Collections	Pending list, record payments
Collection Report	Month-wise summary
All Complaints	List, update status, assign vendor

9. API Request/Response Examples

POST /expenses

json

// Request

{

```
"category": "electricity",  
"expenseDate": "2026-06-01",  
"vendor": "Tata Power",  
"amount": 18500,  
"notes": "June bill"  
}
```

```
// Response
```

```
{  
  "success": true,  
  "data": { "id": "uuid", ... }  
}
```

POST /water/readings

```
json
```

```
// Request
```

```
{  
  "flatId": "uuid",  
  "currentReading": 12630,  
  "month": "2026-06",  
  "photo": "base64_or_url"  
}
```

```
// Response
```

```
{  
  "success": true,
```

```
"data": {  
  "id": "uuid",  
  "previousReading": 12450,  
  "currentReading": 12630,  
  "usage": 180  
}
```

POST /bills/generate

json

// Request

```
{  
  "month": "2026-06",  
  "dueDate": "2026-06-15",  
  "reserveFund": 200,  
  "specialCharges": 0  
}
```

// Response

```
{  
  "success": true,  
  "message": "Bills generated for 32 flats",  
  "totalAmount": 34272  
}
```

GET /bills/my

json

// Response

```
{  
  "success": true,  
  "data": [  
    {  
      "id": "uuid",  
      "month": "2026-06",  
      "dueDate": "2026-06-15",  
      "total": 1071,  
      "status": "unpaid",  
      "pdfUrl": "/bills/uuid/pdf"  
    }  
  ]  
}
```

POST /payments

json

// Request

```
{  
  "billId": "uuid",  
  "amount": 1071,  
  "mode": "upi",  
  "transactionId": "txn_123456",  
  "paidDate": "2026-06-10"  
}
```

// Response

```
{  
  "success": true,  
  "message": "Payment recorded"  
}
```

POST /complaints

json

// Request

```
{  
  "category": "plumbing",  
  "description": "Kitchen sink leaking",  
  "priority": "high",  
  "photo": "base64_or_url"  
}
```

// Response

```
{  
  "success": true,  
  "data": {  
    "id": "uuid",  
    "status": "open",  
    "createdAt": "2026-06-03T10:30:00Z"  
  }  
}
```

PUT /complaints/:id/status

json

// Request

```
{  
  "status": "assigned",  
  "assignedTo": "Plumber Ramesh",  
  "remark": "Will visit tomorrow 10 AM"  
}
```

// Response

```
{  
  "success": true  
}
```

10. Development Timeline

Phase	Hours
-------	-------

Backend	23
---------	----

Frontend	26
----------	----

Total	49 hours (~6.5 days)
--------------	-----------------------------

11. Deliverables Checklist

Backend

- Expenses CRUD APIs
- Water reading submission API
- Water calculation API

- Bill generation API
- Bill PDF generation
- Payment recording API
- Arrears auto-calculation
- Complaints CRUD APIs
- Status update API with notification trigger

Frontend - Resident

- Submit water meter reading with photo
- View water usage history
- View bills list
- Download bill PDF
- Make payment
- Raise complaint
- Track complaint status

Frontend - Admin

- Add/edit/delete expenses
- View all water readings
- Run water calculation
- Generate monthly bills
- View all bills
- Record payments
- View pending collections
- View collection report
- Manage complaints (update status, assign)

Notifications

- WhatsApp on bill generation
- WhatsApp on payment reminder
- WhatsApp on complaint update

Integration (Existing)

- Audit Logs frontend connected
- Timeline Page frontend connected

Document End

